>, <, >=, <=
==

Ls tipos simples: int, float, bool

a = 2 → +, -, *, **, /, //

b = 3.5 → +, -, *, /

c = True → and, or, not

True and False => False
True and True => True
False and False => False

res = 5<8 and 7>10
5<8 → T
7>10 → F
T and F => False

True or False => True
False or False => False

res1 = 0<2 or 3>4
0<2 → T
3>4 → F
T or F => T

print(res1) → True

a = 2
a = float(a) => 2.0
print(a) => 2.0

b = 3.5
b = int(b) => 3
print(b) => 3

print(int(b)) => 3
print(b) => 3.5

bool(0) => False
bool(1) => True
bool(2) => =

int(False) => 0
int(True) => 1

a = 2
type(a) => int

b = 5.66
type(b) => float

x = None
type(x) => NoneType

None

ch = "python"
ch2 = "23"

ch = 'python'
ch2 = '23'

ch3 = " L'étudiant "
ch3 = " L'etudiant "

int(ch2) => 23

int(ch) => Erreur

ch3 = "5.6"

float(ch3) => 5.6

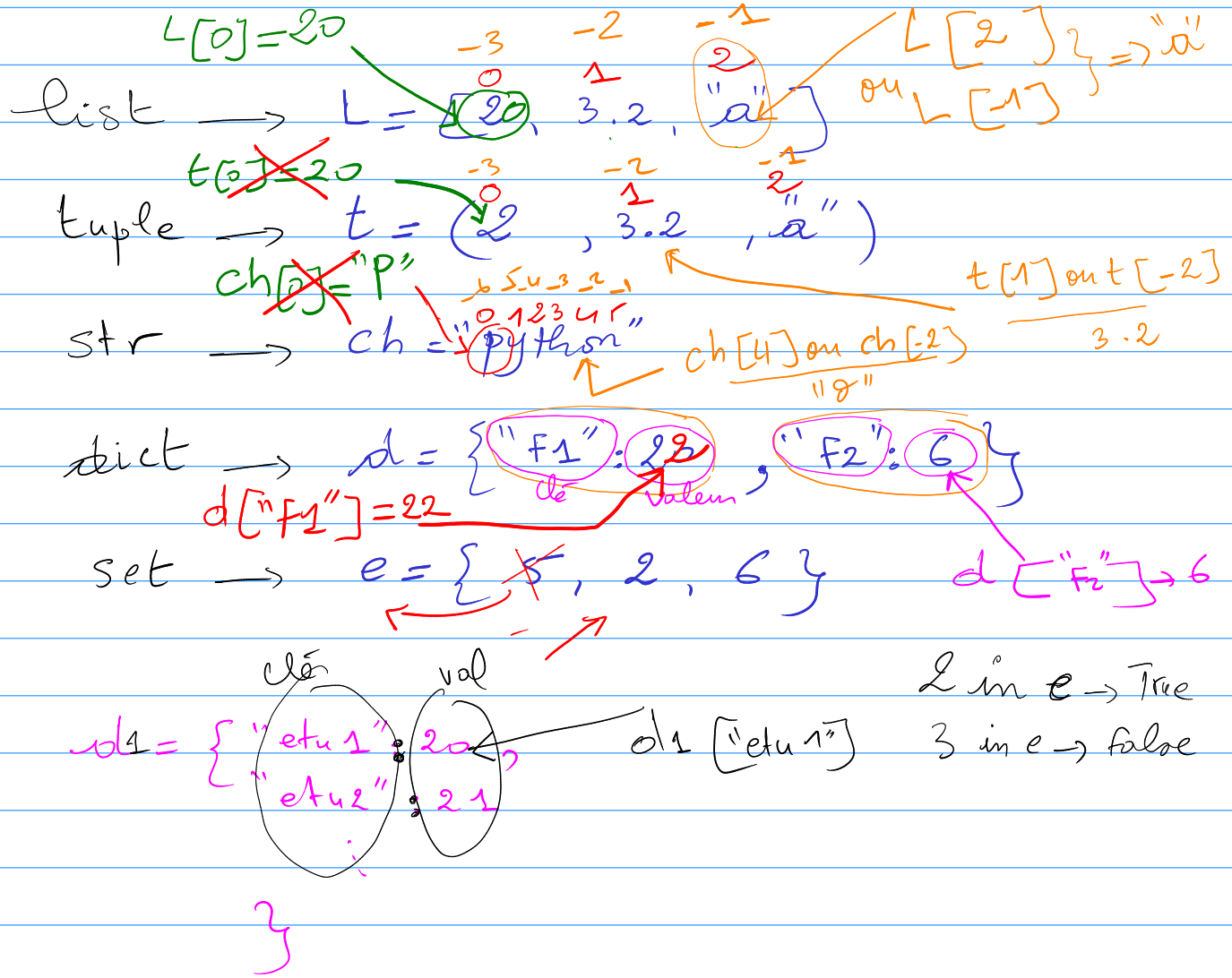
float(ch2) => 23.0

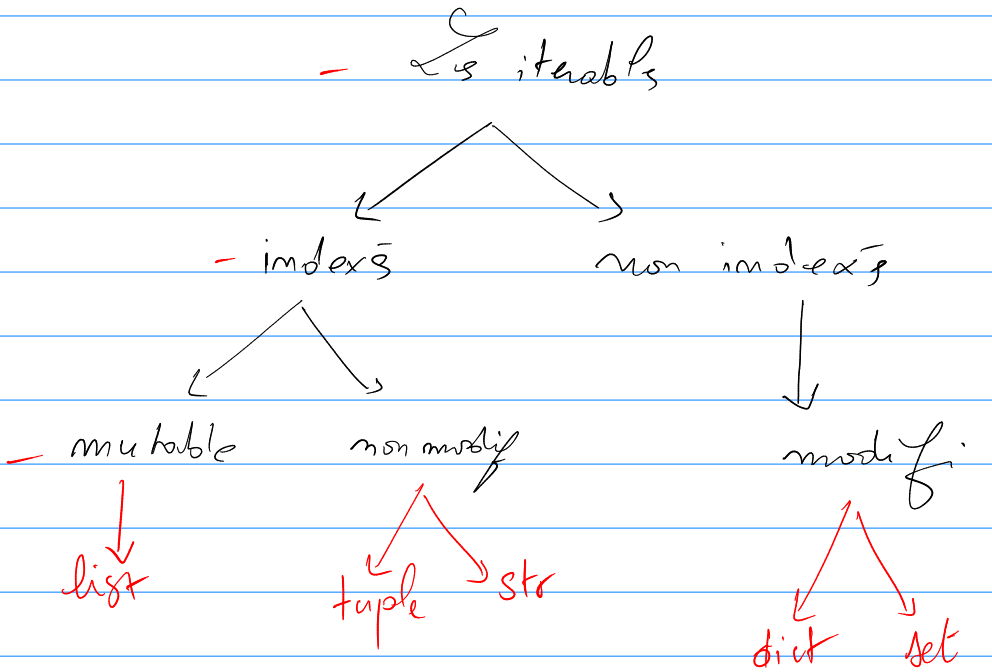
a = 2
b = 3.3

print(a, b) ⟶ 2 3.3

print(f" a = {a} et b = {b} ") ⟶ a = 2 et b = 3.3

Les conteneurs => Les itérables => Les séquences





Opération	`list`	`tuple`	`str`	`dict`	`set`

Création	`L = [1, 2, 3]`
	`L = []`
	`L = list(it)`
	`L = [x*2 for x in it]`
		`T = (1, 2, 3)`
		`T = ()`
		`T = (1,)`
		⚠ virgule
		`T = tuple(it)`
			`s = "abc"` ou `'abc'`
			`s = """multi"""`
			`s = str(42)`
			`s = f"{x}"`
				`d = {"a": 1}`
				`d = {}`
				`d = dict(a=1)`
				`d = {k: v for k, v in it}`
					`S = {1, 2, 3}`
					`S = set()` ⚠ (`{}` = dict)
					`S = set(it)`
					`S = {x for x in it}`

Ajout	`L.append(x)`
	`L.insert(i, x)`
	`L.extend(it)`
	`L += it`
		❌ immuable
		`T = T + (x,)`
		(nouvel objet)
			❌ immuable
			`s = s + "x"`
			`"".join(liste)`
				`d[k] = v`
				`d.update(d2)`
				`d.setdefault(k, v)`
				`d |= d2` (3.9+)
					`S.add(x)`
					`S.update(it)`
					`S |= S2`

Suppression	`L.remove(x)` *(1re occurrence)*
	`L.pop()` / `L.pop(i)`
	`del L[i]` / `del L[i:j]`
	`L.clear()`
		❌ immuable
		`T = T[:i] + T[i+1:]`
		`del T` *(variable entière)*
			❌ immuable
			`s.replace("x", "")`
			`s.strip()` / `lstrip()` / `rstrip()`
				`del d[k]`
				`d.pop(k[, défaut])`
				`d.popitem()` *(dernier inséré)*
				`d.clear()`
					`S.remove(x)` *(KeyError si absent)*
					`S.discard(x)` *(sans erreur)*
					`S.pop()` *(arbitraire)*
					`S.clear()`

Slicing	✅ `L[i:j:pas]` `L[::-1]` (inversion) Affectation : `L[1:3] = [a, b]`	✅ `T[i:j:pas]` Lecture seule	✅ `s[i:j:pas]` `s[::-1]` Lecture seule	❌ (pas d'indices) Accès : `d[k]`, `d.get(k)`	❌ (non ordonné, pas d'indexation)

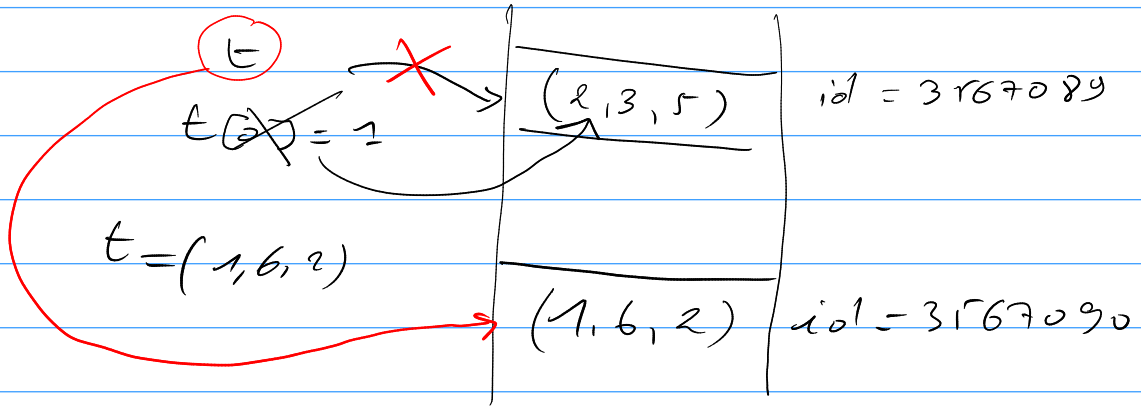
Opérations spécifiques	`L.sort()` / `sorted(L)`
	`L.reverse()`
	`L.index(x)` / `L.count(x)`
	`L.copy()`
	`L * 3`
		`T.index(x)`
		`T.count(x)`
		Déballage : `a, b = T`
		Hachable → clé de dict
			`s.split()` / `sep.join(L)`
			`s.upper()` / `lower()`
			`s.find()` / `s.count()`
			`s.startswith()` / `endswith()`
			`s.isdigit()` / `isalpha()`
				`d.keys()` / `values()` / `items()`
				`d.get(k, défaut)`
				`k in d` *(test sur clés)*
				`d.copy()`
				`dict.fromkeys(it, v)`
					`S1 | S2` union
					`S1 & S2` intersection
					`S1 - S2` différence
					`S1 ^ S2` diff. symétrique
					`S1 <= S2` inclusion

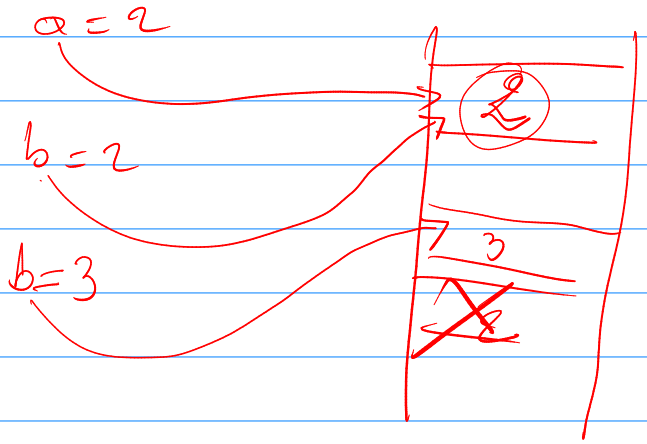
Mutable ?	✅	❌	❌	✅	✅
Ordonné / indexé ?	✅ / ✅	✅ / ✅	✅ / ✅	✅ (3.7+) / ❌	❌ / ❌
Doublons ?	✅	✅	✅	Clés uniques	❌

$t = (2, 3, 5)$ (0, 1, 2)

~~$t[0] = 1$~~

$t = (1, 6, 2)$





L2 = L[0:3] ou L[:3]

L = [2, 3, 5, 0, 10] (0, 1, 2, 3, 4)

L1 = L[2:4]

L[3:5] ou L[3:]

pas

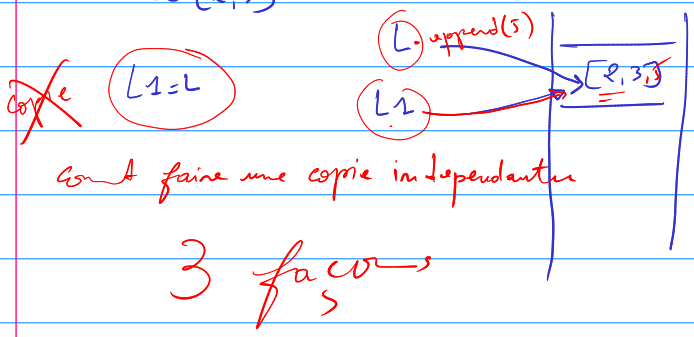
L[::2] ↪ [2, 5, 10]

d[cle] = valeur

ajoute → si la clé n'existe pas

modifie → si la clé existe

L = [2,3]



Comment faire une copie indépendante

3 façons

1. `L2 = L.copy()` ✗
2. `L3 = L[:]` ✗
3. `import copy`
   `L4 = copy.copy(L)`